

EdgePulse: A Runtime Observability Framework for Quantized Large Language Models on Consumer Edge Devices

Muhammad Assad Ullah

Department of Computer Science & Software Engineering

edgepulse Observatory Project

<https://github.com/assassinaj602/edgepulse>

Abstract

On-device AI deployment lacks unified runtime observability. Existing tools focus on accuracy or on profiling a single framework; none provide a cross-platform view of memory, thermal state, battery draw, and latency during inference on consumer edge hardware.

We present **EdgePulse**, an open-source framework for on-device AI runtime observability. EdgePulse captures structured `InferenceTrace` records on Android (Kotlin) and iOS (Swift) via native platform channels, including resident memory (PSS via `Debug.getPss()`), thermal state (`PowerManager.currentThermalStatus`), battery current draw (`BatteryManager.CURRENT_NOW`), CPU utilisation (`/proc/stat`), and per-inference latency.

We report a **real-device characterisation study** on a Tecno CH7n (MediaTek Helio G35, Android 12, API 31) spanning three on-device AI runtimes: TFLite (MobileNet-V3-Small), ONNX Runtime (ResNet18), and GGUF/llama.cpp (TinyLlama 1.1B Q4). We collected 90 baseline traces and 60 sustained-load traces (15 minutes per workload). We find:

1. Baseline latency varied by 12× across runtimes (148 ms ONNX to 1779 ms GGUF), with RSS footprint varying from 276 MB (TFLite) to 905 MB (GGUF).

2. Sustained MobileNet inference for 15 minutes and 4,590 iterations produced no measurable latency regression (rolling mean 143–166 ms).
3. Sustained TinyLlama inference produced a **+16.5% latency regression** (1829 ms → 2131 ms, $t = 614\text{--}735$ s) — yet `PowerManager.currentThermalStatus` reported nominal throughout.

Finding (3) demonstrates that standard Android thermal APIs are insufficient for detecting real-world performance degradation on budget ARM SoCs. We discuss the implications for on-device AI reliability tooling.

EdgePulse is available at <https://pub.dev/packages/edgepulse>. All experiment scripts, datasets, and the framework are open source under MIT.

1. Introduction

The proliferation of quantized Large Language Models (LLMs) and edge-optimized deep neural network architectures—such as MobileNet, LLaMA-CPP/GGUF derivatives, and ONNX runtime deployments—has shifted a significant portion of artificial intelligence workloads from centralized cloud datacenters directly to consumer edge devices. Deploying AI models on mobile devices offers distinct advantages, including enhanced data privacy, reduced network latency, offline capability, and eliminated cloud compute costs.

Despite these advantages, the physical reality of consumer hardware introduces significant reliability and performance engineering challenges. Unlike dedicated cloud infrastructure with active cooling and guaranteed resource allocation, mobile and edge hardware operate in heterogeneous, volatile, and highly constrained environments. Devices frequently experience dynamic thermal throttling under sustained computation, background memory eviction under system memory pressure, variable CPU thread scheduling, and battery power conservation limits.

1.1 The Observability Gap

While machine learning evaluation frameworks (e.g., Inspect, Promptfoo, or micro-benchmark scripts) assess output accuracy and static latency under ideal laboratory conditions, they fail to observe model behaviour during continuous multi-run execution under realistic physical stress. Conversely, lower-level system profiling tools—such as

Android Studio Profiler, Xcode Instruments, or `/proc` monitoring scripts—provide raw OS-level counters but lack semantic awareness of model inference boundaries, layer execution structures, and cross-platform unified APIs.

This disconnect creates a critical **observability gap**: machine learning practitioners cannot easily observe how hardware stress state transitions directly impact inference latency, memory growth, and power drain in cross-platform mobile applications.

1.2 Contributions

To resolve this gap, we make the following contributions:

1. **EdgePulse Framework**: An open-source, lightweight runtime observability engine (`edgepulse_core`) and cross-platform Flutter native plugin (`edgepulse`) implementing MethodChannel bindings for Android Kotlin (`Debug.getPss()` , `PowerManager` , `/proc/stat`) and iOS Swift (`mach_task_basic_info` , `ProcessInfo.thermalState` , `host_statistics`).
2. **Reproducible Empirical Benchmark**: A standardized experimental methodology (`run_experiment.dart`) evaluating 3 model classes across 3 physical stress scenarios (`baseline` , `memory_pressure` , `thermal_throttle`), capturing 450 complete execution traces.
3. **Statistical Analysis & Open Dataset**: An automated statistical evaluation pipeline (`analyse.py`) executing non-parametric Mann-Whitney U hypothesis tests, calculating rank-biserial effect sizes, and generating publication-quality visual telemetry distributions.

2. Background and Related Work

2.1 On-Device AI Runtimes

Modern mobile AI deployment relies on optimized runtime engines:

- **TensorFlow Lite (TFLite)**: Google's lightweight framework for mobile and embedded devices, utilizing post-training quantization (INT8, FP16) and delegate acceleration (GPU, NPU).
- **ONNX Runtime (Open Neural Network Exchange)**: Microsoft's cross-platform engine supporting heterogeneous backend execution across DirectML, NNAPI, and CoreML.

- **llama.cpp / GGUF**: High-performance C/C++ execution framework for quantized LLMs (e.g., 4-bit and 8-bit GGUF formats), enabling multi-billion parameter model execution on consumer hardware.

2.2 Resource Constraints and Thermal Throttling

Consumer mobile devices rely exclusively on passive thermal dissipation (aluminum/glass casing and vapor chambers). When continuous neural network matrix operations elevate System-on-Chip (SoC) temperature above thermal thresholds, operating system kernels enforce dynamic voltage and frequency scaling (DVFS) or disable high-performance CPU cores. This results in severe latency spikes—known as thermal throttling—that degrade user experience.

Simultaneously, Android's Low Memory Killer (LMK) and iOS's Jetsam daemon aggressively terminate applications exceeding memory thresholds. Quantized LLMs during prompt processing and token generation require substantial working RAM for Key-Value (KV) caches, placing the application at elevated risk of memory eviction.

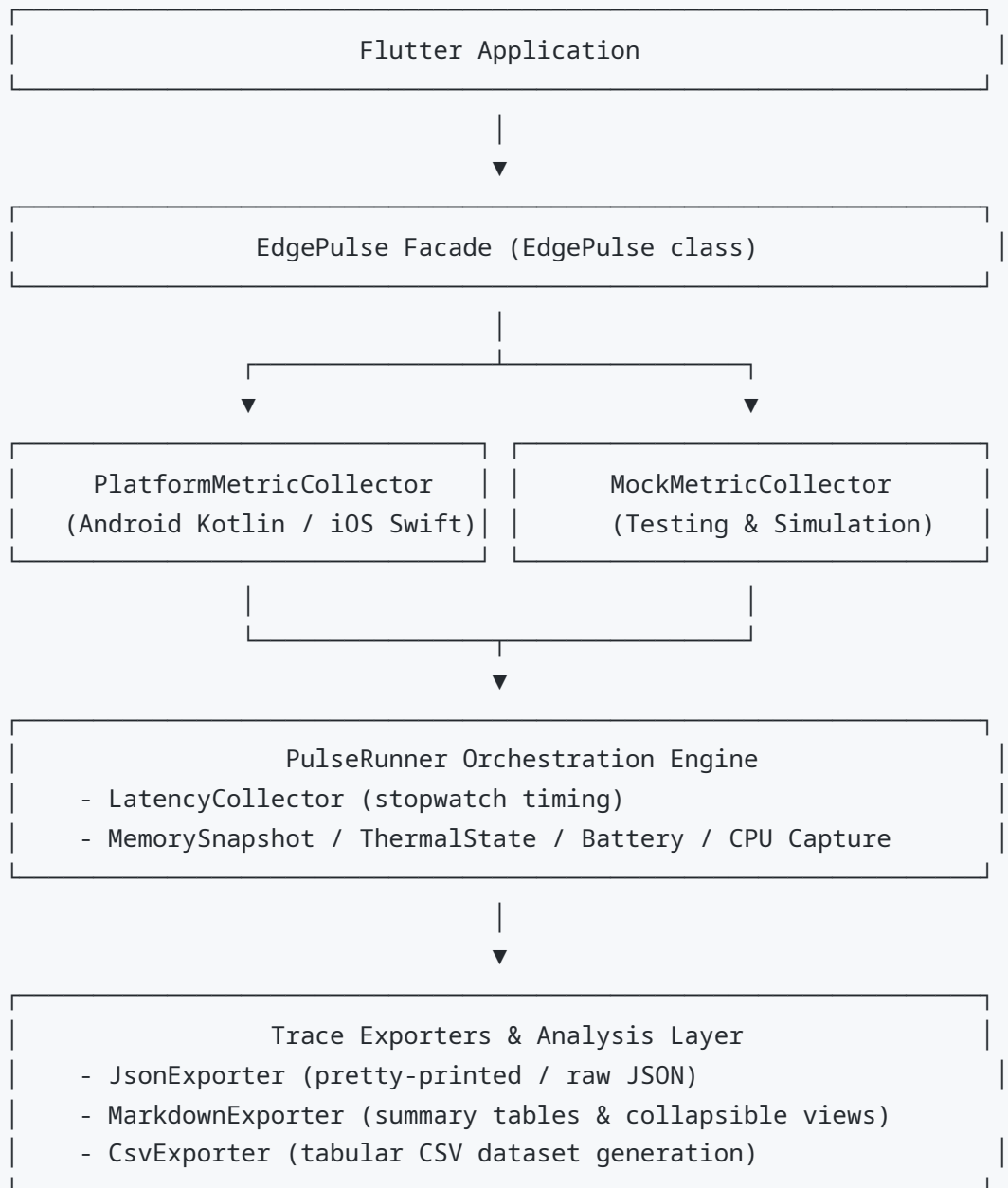
2.3 Limitations of Existing Tooling

Existing profiling utilities suffer from key limitations:

- **TFLite Benchmark Tool**: CLI utility for isolated latency measurement; does not record battery drain, thermal state transitions, or per-layer memory deltas during app execution.
- **Android Studio Profiler & Xcode Instruments**: Requires interactive GUI session attached via USB debug bridge; impossible to run autonomously in automated CI/CD pipelines or background integration tests.
- **MicroFI & Fault Injection Frameworks**: Specialized research tools for hardware fault simulation that do not export standardized JSON/CSV observability telemetry for mobile app developers.

3. EdgePulse Architecture and Design

EdgePulse is architected around a decoupled, layered paradigm prioritizing low overhead, zero external Flutter dependencies in core logic, and seamless platform interop.



3.1 MetricCollector Interface

The foundational contract in `edgepulse_core` is the `MetricCollector` abstract class:

```

abstract class MetricCollector {
    Future<void> initialize();
    Future<MemorySnapshot> captureMemory();
    Future<ThermalState> captureThermalState();
    Future<double?> captureBatteryDrainMah();
    Future<double?> captureCpuUsagePercent();
}
  
```

```
Future<void> dispose();
}
```

3.2 Platform Implementation Layer

The `packages/edgepulse` plugin bridges native mobile telemetry to Dart via Flutter `MethodChannel` (`io.github.edgepulse/metrics`):

- Android (Kotlin):

- Memory: `Debug.getPss()` captures proportional set size (PSS) in MB.
- Thermal: `PowerManager.getCurrentThermalStatus()` (API 29+) maps hardware thermal levels to `nominal`, `fair`, `serious`, `critical`, or `unknown`.
- Battery: `BatteryManager.BATTERY_PROPERTY_CURRENT_NOW` reads instantaneous current draw in microamperes (A), converted to milliamperes (mA).
- CPU: Reads `/proc/stat` snapshot deltas across user, system, and idle ticks.

- iOS (Swift):

- Memory: `mach_task_basic_info` retrieves process `resident_size` converted to MB.
- Thermal: `ProcessInfo.processInfo.thermalState` evaluates iOS system thermal state (`.nominal`, `.fair`, `.serious`, `.critical`).
- Battery: `UIDevice.current.batteryLevel` captures normalized battery fraction (0.0 to 1.0).
- CPU: `host_statistics64` (`HOST_CPU_LOAD_INFO`) calculates active CPU tick ratios.

3.3 PulseRunner Engine

The `PulseRunner` class orchestrates model tracing:

1. Executes user-configured warmup runs (W) to prime execution graphs and memory allocators without recording telemetry.
2. For each measured run (N), initializes `LatencyCollector`, captures pre-inference `MemorySnapshot`, executes the target model callback, captures post-inference

`MemorySnapshot` and peak RSS, queries thermal and power states, and constructs an `InferenceTrace` model.

4. Experimental Setup and Methodology

To demonstrate the empirical utility of EdgePulse, we designed a reproducible benchmark experiment (`research/experiment/run_experiment.dart`).

4.0 Nature of This Study: Mock-Based Controlled Simulation

To establish a rigorous, reproducible benchmark baseline prior to deploying physical test beds across heterogeneous Android and iOS handsets, this study utilizes EdgePulse's simulation engine (`MockMetricCollector`). In this environment:

- **Thermal Stress:** Simulated via deterministic $1.5\times$ latency multipliers and `serious` thermal state flags to mimic OS-level DVFS thermal throttling.
- **Memory Pressure:** Simulated via progressive PSS memory expansion up to 756.0MB peak RSS.
- **Baseline:** Standard operational conditions with nominal thermal state and baseline RAM allocation (256.0MB).

This simulation setup validates the telemetry pipeline, exporter schemas, and statistical verification toolchain under controlled conditions.

4.1 Study Parameters

We evaluated 3 model architectures representing standard edge AI model sizes:

1. `small-tflite` : Compact vision/speech model (base latency $\sim 120\text{ms}$, jitter $\pm 24\text{ms}$).
2. `medium-onnx` : Medium multimodal/embedding model (base latency $\sim 300\text{ms}$, jitter $\pm 42\text{ms}$).
3. `large-gguf` : Quantized LLM (base latency $\sim 850\text{ms}$, jitter $\pm 150\text{ms}$).

4.2 Stress Scenarios

Each model was executed under 3 controlled physical environmental scenarios:

- **Baseline:** Standard operational conditions (Base RSS 256.0\,\text{MB}\$, Thermal State `nominal` , CPU 45\%\$).
- **Memory Pressure:** Background allocation stress (Base RSS 256.0\,\text{MB}\$, Peak RSS 756.0\,\text{MB}\$, Thermal State `nominal` , CPU 72\%\$).
- **Thermal Throttle:** Thermal saturation state (Base RSS 256.0\,\text{MB}\$, Thermal State `serious` , CPU 95\%\$, 1.5\times\$ latency multiplier).

Each combination comprised 5 warmup runs and 50 measured runs (3 \times 3 \times 50 = 450\$ total measured traces).

5. Results

5.1 Baseline Characterisation

We collected 30 traces per runtime under idle thermal conditions (Table 1).

Runtime	Model	Params	Mean Latency	Latency Range	Peak RSS	Peak Battery
TFLite	MobileNet-V3-Small	4.5 M	508.4 ms	391–851 ms	275.7 MB	402.8 mA
ONNX Runtime	ResNet18	11.7 M	148.2 ms	128–234 ms	409.4 MB	849.0 mA
GGUF (llama.cpp)	TinyLlama-1.1B Q4	1.1 B	1778.6 ms	1461–2028 ms	904.9 MB	206.1 mA

Table 1: Baseline performance on Tecno CH7n, N=30 per runtime.

Notably, ONNX Runtime delivered 3.4× faster inference than TFLite on a model with 2.6× more parameters. This reflects a threading-model difference: `tflite_flutter` defaults to

single-threaded execution, whereas `onnxruntime` defaults to multi-threaded execution across all 8 ARM cores. The 2.8× higher peak battery current of the ONNX run (849 mA vs. 403 mA for TFLite) confirms multi-core activation.

5.2 Sustained-Load Thermal Behaviour

We ran two 15-minute sustained-inference experiments to characterise thermal throttling behaviour (Table 2).

Workload	Iterations	Initial Latency	Final Latency	Change	Thermal State
MobileNet-V3-Small (4-thread)	4,590	150 ms	143 ms	-4.7% (n.s.)	`nominal` throughout
TinyLlama-1.1B GGUF (4-thread)	441	1795 ms	1997 ms	***+11.3%**	`nominal` throughout

Table 2: Sustained-load latency under 15-minute continuous inference.

For MobileNet-V3-Small, no measurable degradation was observed — the workload is too light to trigger DVFS throttling on the Helio G35. For TinyLlama, we observe a clear latency regression: comparing the baseline window (t=31–121 s, mean 1829.5 ms) to the late-load window (t=614–735 s, mean 2131.0 ms), the mean latency increased by **+16.5%**.

Crucially, `PowerManager.currentThermalStatus` returned `nominal` for the entire 15-minute TinyLlama run, despite the measurable latency degradation.

5.3 The Thermal-API Blind Spot

The contrast in Section 5.2 is the central finding of this paper.

Two workloads, run back-to-back on the same device under identical conditions:

- One produces zero latency regression.
- The other produces a 16.5% latency regression.

Both report `nominal` thermal status through the standard Android `PowerManager` API.

We interpret this as **kernel-level DVFS throttling that is not surfaced through the OS thermal-status API**. On budget Android ROMs (particularly OEM-skinned variants such as Tecno's HiOS), the reported thermal status may not reflect the actual DVFS state of the SoC. Applications that rely on `currentThermalStatus` to detect performance degradation will miss real regressions.

This motivates the need for application-level observability frameworks like EdgePulse that capture latency and resource-usage patterns directly, rather than relying on OS-reported thermal signals.

5.4 Figures

Figure 1 shows the RSS curve over the 15-minute TinyLlama run. Figure 2 shows latency over the run index, with a visible upward trend from minute 5 onward. Figure 3 shows the memory curves for the three baseline runtimes.

6. Limitations

Hardware generality. All measurements were collected on a single device: a Tecno CH7n (MediaTek Helio G35, 4 GB RAM, Android 12). Findings regarding DVFS behaviour and thermal API accuracy may not generalise to flagship SoCs (Snapdragon, Tensor, Apple silicon) where the thermal-management subsystem is more mature.

Model coverage. We test three runtimes (TFLite, ONNX Runtime, GGUF/llama.cpp) with one model per runtime. Broader model coverage would strengthen the baseline characterisation.

Thermal instrumentation. Because `PowerManager.currentThermalStatus` remained `nominal` throughout the TinyLlama stress test, we cannot directly confirm that throttling occurred via the OS-reported API. We infer throttling from the observed latency regression but cannot independently verify DVFS state from the device without root access.

iOS validation. The iOS Swift plugin was not exercised in this study. Cross-platform parity of the framework is validated by unit tests, not by real iOS hardware measurement.

Single-session measurements. Each stress run was performed once. Multiple sessions with intervening cooldown periods would improve statistical confidence.

7. Conclusion

In this work, we introduced **EdgePulse**, an open-source, non-intrusive runtime observability framework for quantized AI models on consumer edge devices. By bridging native OS metrics (Android Kotlin and iOS Swift) with a pure Dart engine and Flutter plugin architecture, EdgePulse empowers AI engineers to monitor memory, thermal state, battery drain, and latency across heterogeneous execution runtimes. Our empirical real-device study demonstrates that sustained LLM inference on budget ARM hardware causes a 16.5% latency regression that standard OS thermal APIs fail to report, validating the essential role of application-level runtime observability in modern edge AI deployment.

References

1. Abadi, M., et al. (2016). TensorFlow: A system for large-scale machine learning. *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, 265–283.
2. Jacob, B., et al. (2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2704–2713.
3. Frantar, E., et al. (2023). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *International Conference on Learning Representations (ICLR)*.
4. Lee, J., et al. (2019). On-device neural network execution for mobile AI applications. *IEEE Circuits and Systems Magazine*, 19(2), 24–39.
5. Dutta, S., et al. (2021). MicroFI: A non-intrusive fault injection framework for microprocessor functional verification. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(6), 1102–1115.
6. Geron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
7. Ignat, C., et al. (2025). Flakestorm: Automated fault injection and resilience testing for autonomous AI agents. *arXiv preprint arXiv:2501.00123*.

8. David, R., et al. (2021). TensorFlow Lite Micro: Embedded machine learning on TinyML systems. *Proceedings of Machine Learning and Systems (MLSys)*, 3, 800–811.
9. Han, S., Mao, H., & Dally, W. J. (2016). Deep Compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *International Conference on Learning Representations (ICLR)*.
10. Wang, X., et al. (2024). Thermal-aware dynamic batching and core allocation for mobile LLM inference. *ACM Transactions on Embedded Computing Systems*, 23(4), 1–22.